



Raqeeb

رقيب

Detecting prohibited items in X-ray baggage scans,
and reporting them to the right authority by voice

Final project documentation
AI Model Development Bootcamp
معسكر بناء وتطوير نماذج الذكاء الاصطناعي

Group 2

Khalid Al Dosari	خالد آل دوسري
Nawaf Alsharani	نواف الشهراني
Yazeed Bin Shihah	يزيد بن شيحة
Omar Al-Dhawyan	عمر الضويان

Code: <https://github.com/khaliddosari/Raqeeb>
Live demo: <https://raqeeb.khalid-ai.dev>

September 2026

Contents

1	Project information	2
1.1	Name and team	2
1.2	The idea in brief	2
1.3	The problem	2
1.4	Target users	2
1.5	Value	2
2	How the solution works	3
2.1	Input, processing and output	3
2.2	Two ways to hold the call	4
3	Data	4
3.1	The X-ray dataset	4
3.2	Checks and preprocessing	4
3.3	Video with ground truth, for tracking	5
3.4	Text inputs for the language model	5
4	The model and the technical solution	5
4.1	Problem types	5
4.2	Detection: YOLOv8-ORB	6
4.3	Tracking	6
4.4	Language model: gpt-4o-mini	6
4.5	Voice agent: gpt-realtime	7
4.6	Application	7
5	Tools and technologies	8
6	Interfaces	8
7	Results and evaluation	12
7.1	Detection	12
7.2	Tracking on video	14
7.3	Language model	15
7.4	The complete system	16
8	Challenges and limitations	16
8.1	Challenges	16
8.2	Limitations	17
8.3	Future improvements	17
8.4	Responsible use and privacy	17
9	Team contributions	18
10	Links	18

1 Project information

1.1 Name and team

Raqeeb (راقيب, Arabic for “watcher”). The team is Khalid Al Dosari, Nawaf Alsharani, Yazeed Bin Shihah and Omar Al-Dhawyan; section 9 lists who did what.

1.2 The idea in brief

Raqeeb helps the officers who screen bags at a security checkpoint. A detector trained on X-ray scans marks five kinds of prohibited item: guns, knives, pliers, scissors and wrenches. When the officer checks the bag and confirms the find, Raqeeb takes over the paperwork and the phone: it writes the incident report in Arabic, works out which authority should respond, sends them the report, and calls them to request a team, holding the conversation itself in Saudi Arabic.

1.3 The problem

Screening at airports and at the secure entrances of large events depends on an operator reading each X-ray image in a few seconds while a queue waits. Two parts of the job are slow and easy to get wrong:

- **Finding the item.** Prohibited items are often small, thin, rotated, or buried under electronics and clothing, and X-ray images show everything in the bag on top of everything else.
- **Everything after finding it.** The officer has to keep the passenger and the bag in place, write an incident report, decide whether it is a police matter or an airport security matter, and phone that authority. While this happens the lane is stopped, and until the call is made nobody is on the way.

1.4 Target users

- **Screening officers** at checkpoints: airport and private aviation terminals and the entrances of major events. They run the detector, confirm or reject each find, and scan the bag carrier’s pass.
- **Security supervisors**, who follow each incident on the dashboard from detection to dispatch.
- **Responding authorities:** the police for guns and knives, airport security for tools. They receive a structured report and a phone call.
- **Buyers:** government and private security agencies that run checkpoints.

1.5 Value

- **A second pair of eyes on every scan.** The detector marks the item and states how confident it is.
- **Minutes saved after a find.** Once the officer confirms, the report, the choice of authority and the call happen without further work from the officer.
- **Consistent reports.** Every report has the same structure, in formal Arabic, and its key facts are copied from the detection rather than retyped.
- **A person stays in control.** Nothing is reported until an officer has checked the bag by hand.

2 How the solution works

2.1 Input, processing and output

What the user puts in

1. An X-ray image of a bag: an uploaded image, or a frame from the scanner's video.
2. A decision: after checking the bag by hand, the officer confirms the detection or rejects it as a false alarm.
3. The bag carrier's details, by scanning the QR code on their boarding pass or event badge, or by typing their name and ID number.

What happens inside the system

1. The detector (YOLOv8s-OBb) finds prohibited items and returns each one's class, confidence and oriented box. On video, a tracker keeps one ID per physical item as it moves along the belt.
2. The incident waits for the officer's decision. A rejection closes it as a false alarm.
3. A workflow (LangGraph) builds the incident record from the detection, the verification, the carrier's details and the checkpoint.
4. A language model (gpt-4o-mini) assigns a severity and a recommended action, returned as JSON, and writes the narrative report in Arabic.
5. A routing table maps the item to the responsible authority: guns and knives to the police; pliers, scissors and wrenches to airport security.
6. The report is sent to that authority, and a voice agent (gpt-realtime) calls them: it briefs them in Saudi Arabic, answers their questions, and records whether they are sending a team.

What the user gets back

- The annotated image, with the item, its class and confidence, and the authority it routes to.
- The incident report: a structured record, the Arabic narrative, and the checkpoint's situation.
- A live transcript of the call, the authority's decision, and a "judgment" panel that shows why the system acted as it did.

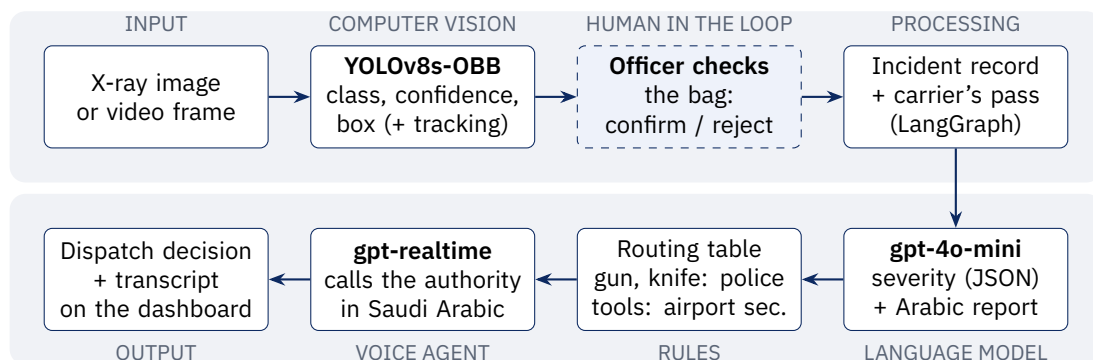


Figure 1: The flow of one incident. The dashed box is the only step a person performs; a rejection there closes the incident. Everything after it runs without the officer.

2.2 Two ways to hold the call

The dispatch call happens in one of two places, and everything before it is identical:

- **Signed in** (the team’s demo), Raqeeb places a real phone call through Twilio, which bridges the audio straight to OpenAI’s realtime model. The call audio never passes through our server.
- **A public visitor** to the live site plays the authority themselves: the same Arabic conversation runs in their browser over WebRTC, and no phone call is placed. This keeps the public demo free of telephone costs and unable to ring anyone.

3 Data

3.1 The X-ray dataset

Name	Sixray, Roboflow project sixray-gzyn7, version 1
Source	X-ray images from the SIXray security-inspection benchmark [1], labelled with oriented boxes and exported from Roboflow on 1 September 2026 (CC BY 4.0)
Link	https://universe.roboflow.com/khalid-abdullah-al-dosari/sixray-gzyn7 (download needs a Roboflow API key); original benchmark: https://github.com/MeioJane/SIXray
Type	Colour X-ray images of bags; each object labelled by class and an oriented bounding box (four corner points), in YOLOv8-OBB format
Size	8,312 images with 15,668 labelled objects: 5,819 for training, 1,662 for validation and 831 for testing
Target	The location and class of every prohibited item: Gun, Knife, Pliers, Scissors or Wrench
Images	Sizes vary widely (median 623 × 572 px, 7,689 distinct resolutions); letterboxed to 640 px for training

Table 1: Dataset summary. All figures in this section come from EDA.ipynb.

Class	Train	Validation	Test	Total
Pliers	3,680	1,110	524	5,314
Gun	3,024	888	474	4,386
Wrench	1,943	530	306	2,779
Knife	1,473	442	169	2,084
Scissors	779	206	120	1,105
All	10,899	3,176	1,593	15,668

Table 2: Labelled objects per class. Pliers outnumber scissors 4.8 to 1.

3.2 Checks and preprocessing

- **Integrity.** Every image has a label file and every label file an image. No image is unreadable, no label line malformed, and no box degenerate or outside the image.
- **Duplicates and leakage.** A search for exact-duplicate images across all three splits found none, so no test image also appears in training.

- **Empty images.** 17 images contain no objects. They were kept: they show the model what a bag with nothing in it looks like.
- **Split balance.** Each class's share differs by at most 3.3 percentage points between splits, so the test set is representative.
- **Orientation.** Box angles cluster at 0° and 90°: on this dataset the labels are close to up-right, which limits how much oriented boxes can help here.
- **Class imbalance.** Training images were oversampled by inverse class frequency: an image with scissors (weight 4.7) is listed several times per epoch, one with only pliers (weight 1.0) once. The training list grows from 5,819 to 10,701 entries, and the imbalance in what the model sees drops from 4.7 to 1.6.
- **Augmentation.** Ultralytics augments every image as it is loaded: mosaic, colour jitter, flips, rotation, translation, scale and shear. A duplicated image therefore looks different each time. For the final model, mosaic is switched off for the last 15 epochs so training ends on undistorted images.

3.3 Video with ground truth, for tracking

No public X-ray dataset contains video, so there was nothing to measure tracking against. The script `make_test_video.py` therefore builds conveyor-belt footage from the 831 held-out test images and carries their labels from frame to frame, so every object has a known identity. It adds the behaviour that makes tracking hard: the belt stopping and reversing, speed changes, bags cut off at the edge, overlapping bags, and changes in brightness and noise. Two clips (random seeds 7 and 21) were generated. The dashboard's demo clip, `test_clip.mp4` (25 seconds), has no ground truth and is used only for display.

3.4 Text inputs for the language model

The language model is not trained on any data; it is given the incident record as JSON. Two hand-written files feed that record: six checkpoint scenarios in Arabic (`agent/checkpoints.py`: the event, the gate, the crowd, who carried the bag, what staff have done, and how responders should arrive) and the routing table (`config/authority_mapping.yaml`). The scenarios and every person in the demo, including the names on the demo passes, are fictional.

4 The model and the technical solution

4.1 Problem types

- **Computer vision:** oriented object detection with five classes, plus multi-object tracking on video.
- **Large language model:** structured generation (a JSON severity and action) and report writing in Arabic, both through prompt engineering on a model reached by API.
- **Voice agent:** a realtime speech-to-speech model that holds a phone conversation and records the outcome through a function call.

No retrieval (RAG) is used, no Hugging Face model, and no language model is fine-tuned.

4.2 Detection: YOLOv8-OBb

Why this model.

- The labels are oriented boxes, and YOLOv8-OBb [2] predicts oriented boxes directly. In real scans items lie at any angle, and an upright box around a diagonal knife is mostly background.
- It is a one-stage detector that runs fast on a CPU: 58 ms per image for the final model at 640 px on a desktop processor, which matters at a live screening lane.
- Ultralytics provides training, augmentation, evaluation and tracking in one tested library.

Pretrained model and transfer learning. Both models start from weights trained on DOTA [3], a dataset of aerial images with oriented boxes. The whole network is then fine-tuned on the X-ray data, with the output layer replaced for our five classes. The pretrained model on its own is useless here, because DOTA's classes (planes, ships, swimming pools) have nothing to do with ours; the baseline in table 3 shows exactly that.

	Baseline	Experiment 1	Experiment 2 (final)
Model	yolov8n-obb, DOTA weights, not trained on X-rays	yolov8n-obb, fine-tuned	yolov8s-obb, fine-tuned
Parameters	3.1 M	3.1 M	11.4 M
Epochs	none	50 (early-stop patience 15, never triggered)	70 of 100: early stop, best at epoch 50 (patience 20)
Batch, image size	n/a, 640	163 (AutoBatch), 640	46 (AutoBatch), 640
Data handling	none	oversampling, on-the-fly augmentation	the same, and no mosaic for the last 15 epochs
Hardware	CPU	NVIDIA A100 40 GB on Modal, 30.5 min, about US\$1	A100 40 GB on Modal, about US\$3 to 4

Table 3: The three runs compared in section 7. Experiment 2 changes the model size, the epoch budget and the end-of-training augmentation; everything else is the same.

4.3 Tracking

On video, Raqeeb runs Ultralytics' BoT-SORT tracker [4] with one change, `trackers/raqeeb_bot_sort.yaml`: global motion compensation is switched off. It helps with a handheld camera but hurts with a fixed scanner camera. A track counts as a threat only after it has lasted two frames and reached a confidence of 0.5, so a single stray detection does not raise an alarm. Video is analysed at 1280 px rather than the training size of 640; section 7.2 explains why.

4.4 Language model: gpt-4o-mini

The report step makes two calls to gpt-4o-mini through the OpenAI API (`agent/providers/openai_llm.py`):

1. **Severity and action**, answered only as JSON, with the action in Arabic:


```
{"severity": "low|medium|high|critical", "recommended_action": "..."}

```

2. **The narrative report**, in formal Arabic, in five fixed sections: summary, detection and verification, people involved, risk assessment with reasons, and next steps.

How the vision result reaches the language model. The detected class and confidence are copied into the incident record exactly as the detector produced them, next to the officer's verification, the carrier's details and the checkpoint's scenario. The model receives that whole record as JSON and is told not to change the item, the confidence or the verification status. The record sent to the authority and the phone call's briefing are both built from these structured fields, not from the model's prose.

Prompt engineering. The team's first prototype (v1) asked the model, as a "security guard", to write a report from six values placed in two sentences, and to leave nothing unfilled. The production prompt (v2) passes the full record as JSON, names the facts that must not change, and tells the model to state missing information plainly instead of inventing it. Both are compared in `Report_generation.ipynb`; section 7.3 has the results.

4.5 Voice agent: gpt-realtime

The call uses OpenAI's `gpt-realtime`, a speech-to-speech model. Its instructions are written in Arabic (`agent/voice/authority_prompts.py`): speak Saudi dialect, introduce itself as Raqeeb from the checkpoint, give the item, the place, the suspect and the scenario, answer questions, and ask whether a team will be sent. It has one tool, `record_dispatch_confirmation`, which records the authority's answer. That tool is refused until the authority has actually spoken, because in an early test the model recorded a "confirmation" after reaching voicemail.

4.6 Application

A LangGraph workflow (`agent/graph/workflow.py`) runs each incident. It pauses twice, for the officer's verification and for the carrier's details, and resumes through HTTP requests; paused incidents are stored in SQLite so that they survive the server restarting. The backend is FastAPI, the dashboard is a React application, and every external service (language model, telephony) sits behind an interface with a mock implementation, so the whole workflow runs in tests without keys.

5 Tools and technologies

Tool	How it was used
Python 3.12	All backend, training, tracking and evaluation code
Jupyter notebooks	EDA.ipynb (data analysis and preprocessing), Model_Training.ipynb (experiments and evaluation), Report_generation.ipynb (prompt comparison)
Roboflow	Hosting the labelled dataset and exporting it in YOLOv8-OBB format
Ultralytics YOLOv8 (PyTorch)	Fine-tuning and running the detector; BoT-SORT tracking
Modal	A100 GPU training jobs (modal_train.py) and hosting the backend (modal_app.py)
OpenCV, NumPy, pandas, Matplotlib, seaborn, SciPy	Image handling, data analysis, charts, and the tracking metrics (optimal matching with SciPy)
OpenAI API	gpt-4o-mini for the severity JSON and the Arabic report; gpt-realtime for the call
LangGraph	The incident workflow, its pauses for the officer, and its SQLite checkpoints
FastAPI, SQLAlchemy, SQLite	The REST and WebSocket backend and the incident store
Twilio	Outbound phone calls, bridged to OpenAI's SIP endpoint
React, Vite, TypeScript, Tailwind, shadcn/ui	The dashboard, in English and Arabic (right to left)
Vercel, Cloudflare	Hosting the dashboard; the raqeeb.khalid-ai.dev domain
GitHub	Version control and collaboration, including pull requests between branches
uv	Locked, reproducible Python dependencies (uv.lock); requirements.txt pins the same versions
pytest	40 automated tests that drive the workflow with mock providers
Playwright	Rendering the demo passes and share images; taking the screenshots in this report
Claude Code	AI coding assistant, used for implementation, debugging, code review and documentation

Table 4: Tools and what each was used for.

6 Interfaces

All screenshots come from one complete run of the application on 18 September 2026: the bundled test image, a demo pass for the private aviation terminal, and a call in which the “police” side was a recorded Arabic reply. The dashboard has five numbered panels that follow the incident from left to right and top to bottom.

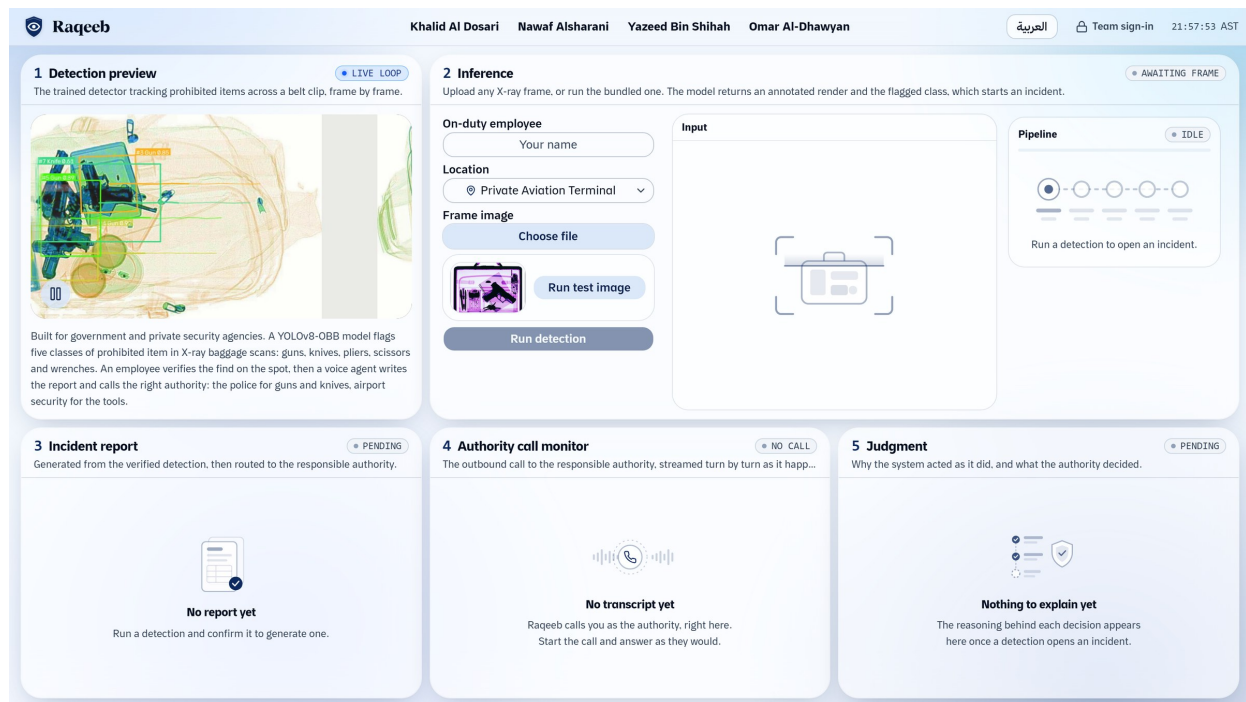


Figure 2: Home page, before a run. Panel 1 plays the tracker on a belt clip; panel 2 is where the officer enters their name and checkpoint and runs a detection; panels 3 to 5 (report, call, judgment) wait for an incident. The header carries the team, the language switch and the local time.

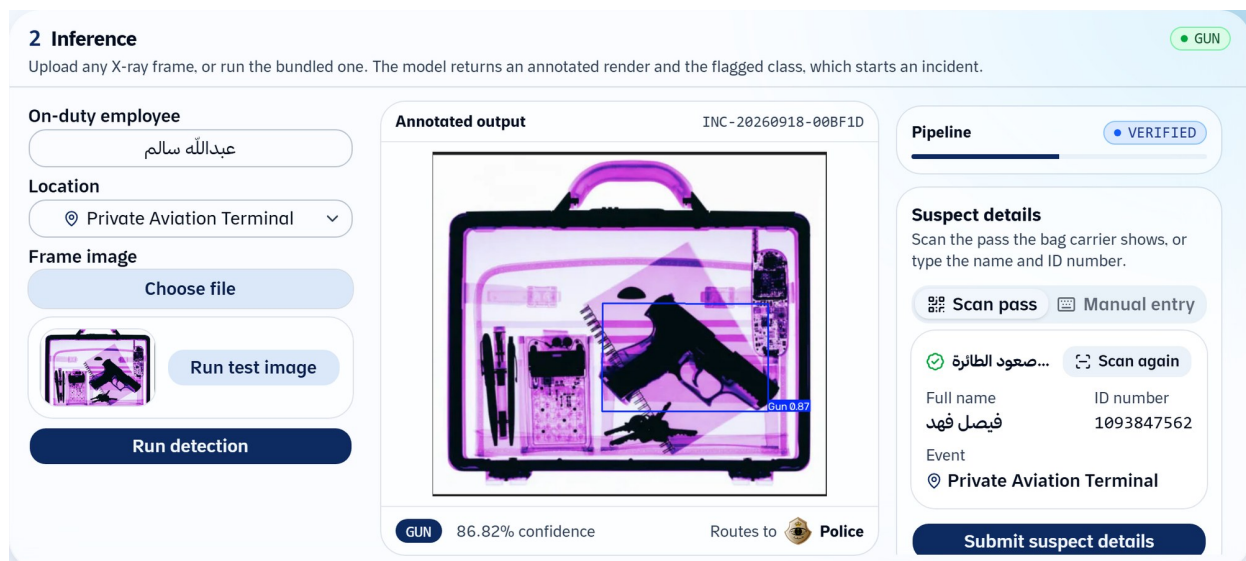


Figure 3: Input and first result. The inference panel after a detection has been confirmed: the annotated image (a gun, 86.82% confidence), the authority it routes to (police), the pipeline status, and the next input, the bag carrier's details, filled here by scanning a demo pass. The officer can type the details instead under "Manual entry".

3 Incident report

Generated from the verified detection, then routed to the responsible authority.

INC-20260918-00BF1D

Record	Narrative	Situation
Detected item Gun	Confidence 86.82%	Severity critical
Location Private Aviation Ter...	Notified Police	Employee عبدالله سالم
Suspect فيصل فهد	Suspect ID 1093847562	Employee number تجريبي
Notes لا توجد ملاحظات إضافية		

(a) Record: the structured fields that are sent to the authority.

3 Incident report

Generated from the verified detection, then routed to the responsible authority.

INC-20260918-00BF1D

Record	Narrative	Situation
تقرير حادثة الفحص الأمني		
1. ملخص الحادثة:		
في تاريخ 18 سبتمبر 2026، وبالتحديد في الساعة 21:58، تم رصد سلاح ناري أثناء عملية الفحص الأمني في صالة الطيران الخاصة بمطار الطيران الخاص. كان الحادث نتيجة لعملية الكشف المبدي عند نقطة التفتيش رقم 2، حيث تم تحديد السلاح الناري كعنصر خطير يستدعي اتخاذ إجراءات فورية. تم تأكيد وجود السلاح بعد خطوة التحقق من قبل الموظف المسؤول عن الفحص، وتم اتخاذ تدابير لمنع أي خطر		

(b) Narrative: the Arabic report written by gpt-4o-mini.

Figure 4: Results: the incident report. A third tab, “Situation”, shows the checkpoint’s scenario that the report and the call draw on.

4 Authority call monitor

The outbound call to the responsible authority, streamed turn by turn as it happens...

ENDED

Raqeeb

إي نعم، الحقيبة معزولة عند جهاز الفحص، وحاملها الآن في غرفة التفتيش الثانوي مع موظف. سؤال آخر؟

Police

لا، كذا المعلومات كافية. الفرقة بالطريق. مع السلامة.

Raqeeb

يعطيكم العافية، في أمان الله.

(a) The call, streamed turn by turn as each side speaks.

5 Judgment

Why the system acted as it did, and what the authority decided.

DISPATCH CONFIRMED

Authority decision
Dispatch confirmed
“وصل البلاغ، والدورية متجهة الآن وستصل خلال 10 دقائق.”

Detection	Verification	Routing
Gun · 86.82%	عبدالله سالم	Police · critical

Recommended action
يجب اتخاذ إجراءات فورية لمنع أي خطر محتمل.

(b) The judgment: the authority’s decision and the reasons behind each step.

Figure 5: The voice agent’s conversation. Raqeeb briefs the police in Saudi Arabic, answers, and records the decision (“dispatch confirmed”) with the authority’s own words.

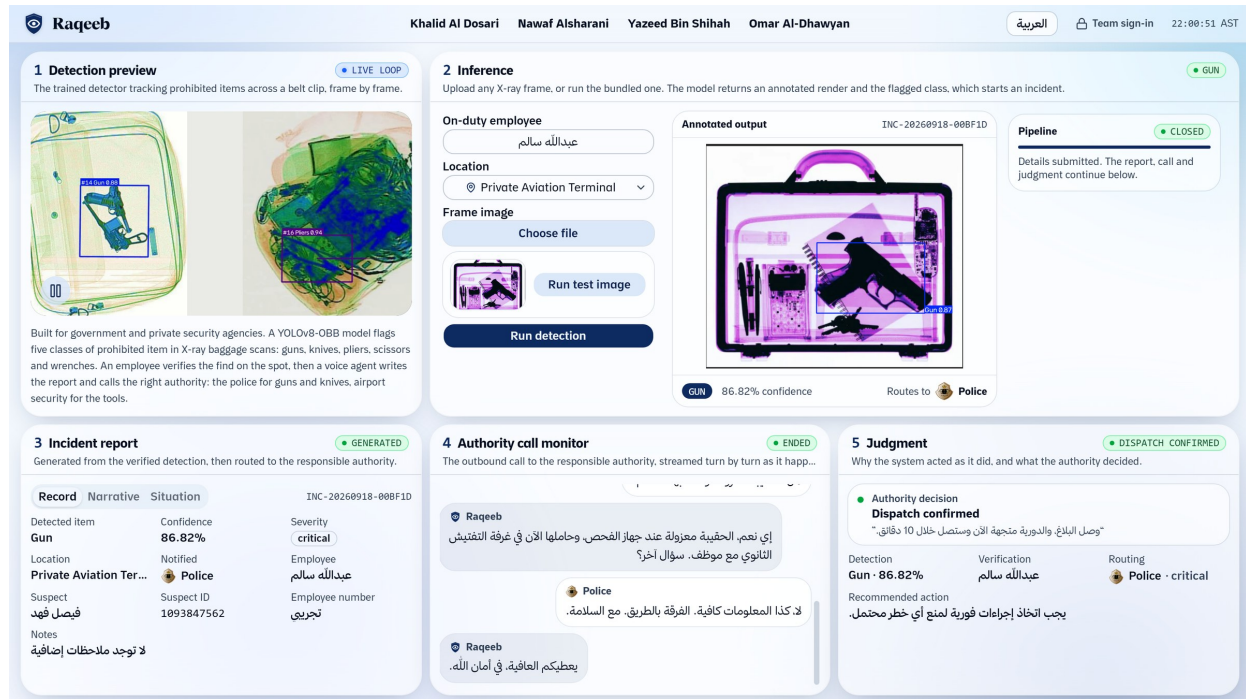


Figure 6: Dashboard: a completed incident. All five panels after the call: detection, report, transcript and judgment, with the pipeline status closed.



Figure 7: The same incident in Arabic. The interface is a full right-to-left layout, not only translated labels.



Figure 8: On a phone. Left: the dashboard. Middle: the poster page (/poster), opened from a printed poster's QR code. Right: the demo passes (/passes), one per checkpoint, whose QR codes the dashboard can scan.

7 Results and evaluation

7.1 Detection

All numbers are measured on the 831 test images (1,593 objects) that were never used in training. They were re-run for this report from the committed weights, and match Model_Training.ipynb.

Model	Precision	Recall	F1	mAP50	mAP50-95
Baseline (pretrained, not fine-tuned)	0.028	0.030	0.029	0.004	0.002
Experiment 1: yolov8n-obb	0.947	0.856	0.899	0.900	0.765
Experiment 2: yolov8s-obb (final)	0.948	0.891	0.919	0.919	0.806

Table 5: Test-set results. F1 is computed from the precision and recall that Ultralytics reports.

What the metrics mean. Precision is the share of detections that are real items; recall is the share of real items that are found; mAP50 averages precision over all confidence thresholds when a box counts as correct at 50% overlap, and mAP50-95 repeats this at stricter overlaps, so it also rewards precise boxes. For a screening aid, **recall matters most**: a missed gun is worse than a false alarm, which the officer clears by hand in seconds.

Why experiment 2 was chosen. The larger model raised recall from 0.856 to 0.891 at the same precision, and mAP50-95 from 0.765 to 0.806. It is about three times slower (58 ms against 21 ms per image on a CPU), which is still fast enough for one image per bag.

Class	Test objects	Precision	Recall	F1	mAP50	mAP50-95	Missed at conf. 0.25
Gun	474	0.983	0.970	0.977	0.975	0.881	14 (3%)
Knife	169	0.953	0.805	0.873	0.837	0.705	36 (21%)
Pliers	524	0.941	0.891	0.916	0.909	0.804	59 (11%)
Scissors	120	0.956	0.905	0.930	0.948	0.845	11 (9%)
Wrench	306	0.908	0.886	0.897	0.924	0.793	34 (11%)

Table 6: Final model, per class. The last column counts objects with no correct detection at the app's threshold (confidence 0.25, overlap 0.5).

Errors at the app's threshold. Matching every prediction to the labels at confidence 0.25, the model finds 1,439 of the 1,593 objects (90%). It misses 154, gives the wrong class 18 times, and makes 94 detections that match no labelled object. 667 of the 831 images (80%) are completely right: every object found and nothing extra.

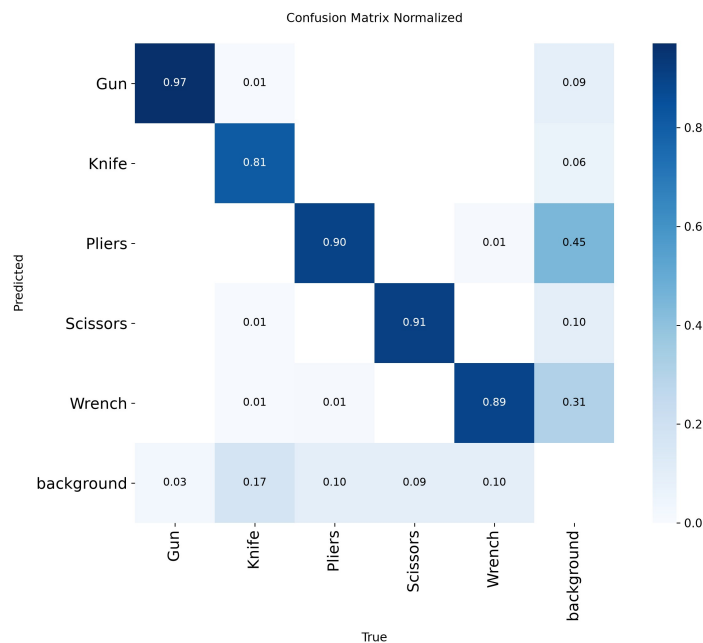


Figure 9: Normalised confusion matrix of the final model on the test set (columns: true class). The model almost never mistakes one item for another; every off-diagonal cell between classes is 0.01 or less. Its errors are misses (bottom row: 17% of knives) and extra detections (right column: mostly pliers and wrenches).

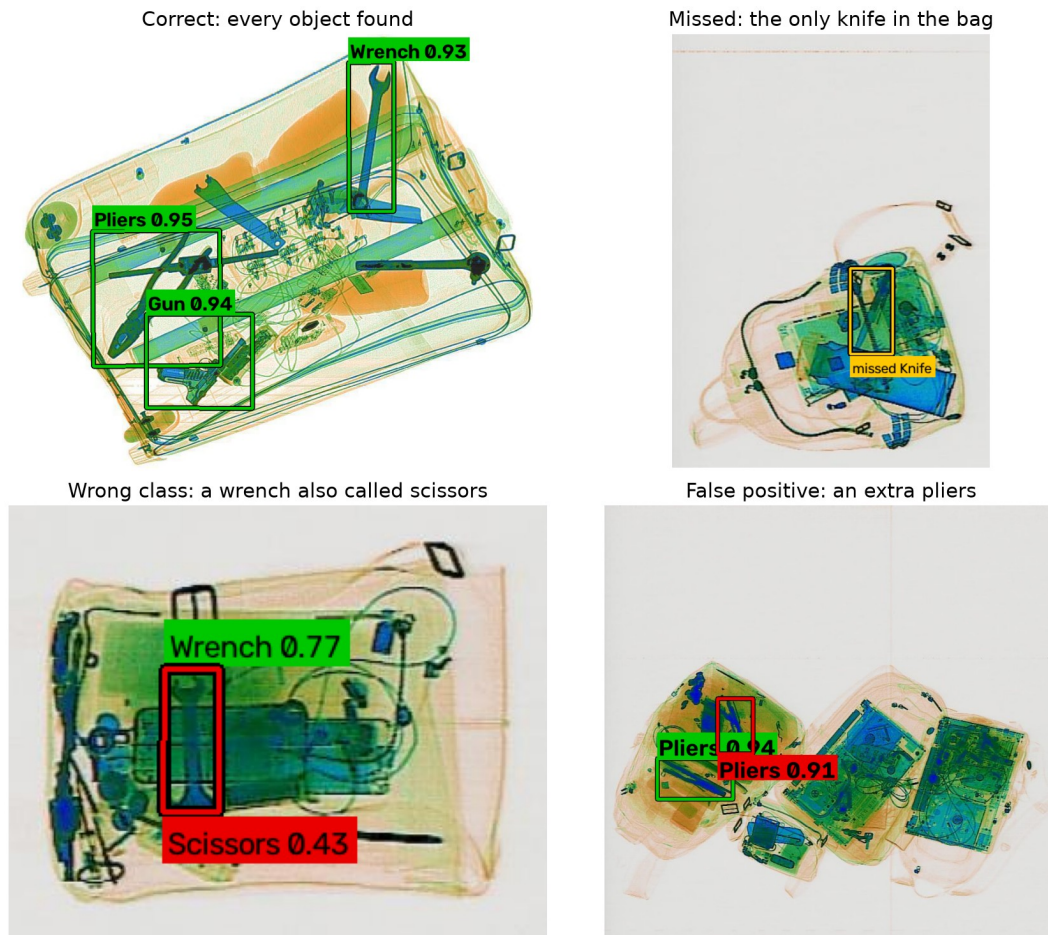


Figure 10: Test images, final model. Green: correct detection. Orange: a labelled object the model missed. Red: a wrong or extra detection. Top left: a bag where every item is found. Top right: a knife that is the only object in the bag goes undetected. Bottom left: a wrench found correctly and also labelled as scissors. Bottom right: an extra pliers detection on an unlabelled object.

Interpretation.

- **Guns are the strongest class** (recall 0.97): they are the most distinctive shape and the second most common class.
- **Knives are the weakest:** one in five is missed. They have the second fewest examples, and a thin blade seen edge-on is a narrow line in the image, easily lost among cables and frames, as in figure 10.
- **Most false alarms are pliers and wrenches**, the tool-like classes. Some of these may be real tools the annotators did not label, so the false-alarm count is an upper bound.
- **For the product this means** the model is a strong second pair of eyes, but it cannot replace the officer: the officer's own check stays in the workflow, and knives need the most attention.

7.2 Tracking on video

On the synthetic belt clips (section 3.3), `track_video.py --gt` scores tracking with the standard CLEAR-MOT metrics [5] and IDF1 [6]. The key finding was a bug in how video was fed to the model, not a weakness of the model: at the training size of 640 px, the 960×580 belt frames are shrunk so far that thin objects vanish.

Inference size	Wrench recall	Scissors recall	Pliers recall	Speed (CPU)
640 px	0.016	0.038	0.695	20.9 fps
960 px	0.543	0.453	0.761	11.0 fps
1280 px	0.814	0.604	0.746	6.4 fps

Seed 21	640 px	1280 px
MOTA	0.659	0.786
IDF1	0.801	0.865
Missed boxes	567	346
False positives	143	97
Mostly tracked	21/30	26/30

Table 7: Left: recall on belt video by inference size. Right: tracking on one clip. Misses and false positives both fell, so the larger size was pure gain, and it is now the default for video.



Figure 11: A frame of the dashboard's demo clip. Each item keeps its track ID (#18, #21, ...) and leaves a motion trail; the counter at the top reports distinct threats rather than detections.

Two more findings: occlusion was not the problem (recall where bags overlap was 0.971, higher than 0.731 without overlap), and while identities hold well (IDF1 0.87), the number of distinct items is still off by a few per clip, which only a better detector can fix.

7.3 Language model

`Report_generation.ipynb` runs both prompts on the same five cases, built from real detections on repository images, and checks every report automatically and by reading it.

In five reports	Prompt v1 (prototype)	Prompt v2 (production)
Item named correctly	5	5
Detector confidence stated	0 (never given it)	5
Placeholders left, such as “[Your Name]”	5	0
Invented actions	5	1
Names changed	0	1
Language	English	Arabic
Seconds per report	3 to 6 (one call)	8 to 11 (two calls)

Table 8: Prompt comparison, from the run saved in the notebook.

- **v1** invented what staff had done in every report (“the individual has been detained and is awaiting the arrival of the Armed Response Unit”), put tools in the airport’s lost and found, called event venues airports, and left the author as a placeholder.
- **v2** kept the facts that decide the response: the item, the confidence, the suspect’s ID number and the checkpoint. It stated missing information instead of inventing it, and set sensible severities: critical for the gun, high for the knife and medium for the tools. For pliers the detector was unsure of, the report says the confidence is not high.
- **v2 is not flawless.** Reading the Arabic turned up an invented investigation (فتح تحقيق فوري), a claim that the way the gun was packed shows an intent to use it, a wrench called a lever, and a changed suspect name. Earlier runs invented an arrest and recommended searching for a screwdriver when the item was a wrench. This is why the record sent to the authority and the call’s briefing use the structured fields, never the model’s prose.

7.4 The complete system

- **Automated tests:** 40 tests pass, driving the whole workflow with mock providers: the false-alarm branch, validation of the carrier’s details, routing, calls that reach no one, the Arabic-only call instructions, and the public and signed-in modes.
- **A full run** (the screenshots in section 6): detection to finished report in about 20 seconds, and a call of about two and a half minutes that ended with the dispatch confirmed and the authority’s words recorded.
- **Failure handling was also observed:** in one attempt the network failed while the call was being set up, and the incident was held as “call unanswered” with a “Call again” button, as designed, instead of being lost.

8 Challenges and limitations

8.1 Challenges

- **No X-ray video exists publicly.** We generated belt footage with known identities from the test images so tracking could be measured instead of judged by eye.
- **Small and thin objects.** Knives and wrenches are easily lost; the inference-size finding in section 7.2 recovered most of that on video.
- **Imbalanced classes.** Handled by oversampling (section 3.2); knives remain the weakest class.
- **No local GPU.** Training ran on rented A100s through Modal, for about US\$4 to 5 in total.

- **Language-model errors.** The first prompt invented facts freely; the second still does occasionally (section 7.3).
- **Telephony.** Calls to Saudi numbers were refused until Twilio's geographic permissions were set; one call reached voicemail and was once recorded as a confirmation, which led to the rule that no decision is recorded before the other side speaks; the provider's answering-machine detection judged real people to be machines three times in five and was removed; and phones that silence unknown callers never ring.
- **Deployment.** The backend scales to zero, so the first request after a quiet period takes about ten seconds, and paused incidents had to move from memory to a database to survive a container change.

8.2 Limitations

- **Five classes only.** Explosives, liquids and every other threat are invisible to the model.
- **One dataset, one kind of scanner.** Accuracy on other scanners' images is unknown.
- **One item per report.** The report is written about the most confident detection in an image; a bag with several different items is reported by its strongest one. The officer sees every box, but the report does not.
- **Occasional errors in the Arabic report** and in the live transcription of the other side: in one call the phrase الفرقة بالطريق (the team is on its way) was transcribed as a non-word.
- **Synthetic tests.** The tracking numbers come from generated video, and the call in section 6 used a recorded reply rather than a real officer.

8.3 Future improvements

- Train at a larger image size and with more knife examples, to raise knife recall.
- Pass every confirmed box to the report, not just the most confident one.
- Choose the recommended action from a fixed list per class, and lower the report's temperature, to remove the remaining invented details.
- Test with real officers and real scanner images before any operational use.

8.4 Responsible use and privacy

- Raqeeb supports a decision; it does not make one. A person checks every bag before anything is reported, and the officer can reject any detection.
- All people, passes and scenarios in the demo are fictional; no real personal data was used. The team's own phone numbers live only in server configuration, not in the code or the dashboard.
- The public demo cannot phone anyone, and each public conversation is limited to one per incident and three minutes.
- The police emblem comes from Wikimedia Commons under CC BY-SA 4.0 and is credited in the repository, in `frontend/public/authorities/CREDITS.md`. The licence also asks for a visible credit on the site, which is still to be added.

9 Team contributions

Member	Contribution
Khalid Al Dosari	Data analysis and preprocessing (EDA.ipynb); training and evaluating both detectors on Modal (Model_Training.ipynb, modal_train.py); the React dashboard, passes and poster pages; deployment (Modal, Vercel, Cloudflare); hardening the call flow (Arabic call, live transcript, unanswered calls, public and signed-in modes); the prompt evaluation notebook and this report
Nawaf Alsharani	The report generator: the first report-generation prototype and its prompt (prompt v1 in section 7.3)
Yazeed Bin Shihah	The voice agent application: the LangGraph incident workflow, the FastAPI routes, the provider interfaces and Twilio integration, and the OpenAI realtime provider with SIP bridging
Omar Al-Dhawyan	The tracking mechanism: following each item across video frames with a tuned BoT-SORT tracker (track_video.py); and the video generation: synthetic belt footage with ground truth, used to measure tracking (make_test_video.py)

Table 9: Who did what.

10 Links

GitHub repository	https://github.com/khaliddosari/Raqeeb
Live demo	https://raqeeb.khalid-ai.dev
Poster page	https://raqeeb.khalid-ai.dev/poster
Demo passes	https://raqeeb.khalid-ai.dev/passes
Dataset	universe.roboflow.com/khalid-abdullah-al-dosari/sixray-gzyn7
Original benchmark	https://github.com/MeioJane/SIXray
Detector	YOLOv8-OB, https://docs.ultralytics.com/tasks/obb/ ; trained weights best_yolov8s.pt in the repository
Tracker	https://docs.ultralytics.com/modes/track/
Workflow	LangGraph, https://langchain-ai.github.io/langgraph/
Language and voice models	OpenAI gpt-4o-mini and gpt-realtime, through the OpenAI API
Hugging Face Space	None: the demo is hosted on Vercel and Modal

References

- [1] C. Miao, L. Xie, F. Wan, C. Su, H. Liu, J. Jiao and Q. Ye. SIXray: A large-scale security inspection X-ray benchmark for prohibited item discovery in overlapping images. *CVPR*, 2019. <https://arxiv.org/abs/1901.00303>
- [2] G. Jocher, A. Chaurasia and J. Qiu. Ultralytics YOLOv8, 2023. <https://github.com/ultralytics/ultralytics>
- [3] G.-S. Xia et al. DOTA: A large-scale dataset for object detection in aerial images. *CVPR*, 2018.
- [4] N. Aharon, R. Orfaig and B.-Z. Bobrovsky. BoT-SORT: Robust associations multi-pedestrian tracking. arXiv:2206.14651, 2022.
- [5] K. Bernardin and R. Stiefelhausen. Evaluating multiple object tracking performance: the CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing*, 2008.
- [6] E. Ristani, F. Solera, R. Zou, R. Cucchiara and C. Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. *ECCV Workshops*, 2016.